

ASSIGNMENT 05
Due date: 2026/05/07 | 05

Aim: Navigation, Deep Links, CompositionLocals & Coroutines

1 Multi-Screen App with Navigation

Design and implement a multi-screen Android application using Jetpack Compose and Material3. The app should feature a fully functional navigation system and a navigation host managing multiple screens.

Technical Hint: Navigation Compose & Data Passing

Use the `NavHost` composable as the central anchor point. To pass an `itemId` from the Home screen to the Detail screen in a type-safe manner, define the argument in the route as `detail/{itemId}`. Avoid passing entire data objects directly in the navigation route; instead, navigate using the ID and load the object, e.g., from a repository.

☐ Implementing Screens:

- ☐ Develop at least three screens (Home, Detail, and Settings).
- ☐ Utilize the Navigation Compose library to handle transitions between these screens.
- ☐ Ensure that parameters (e.g., an `itemId` from the Home screen) can be passed to the Detail screen.

☐ App Bar Navigation:

- ☐ Display the title of the current screen in the AppBar.
- ☐ Include a back button in the AppBar that allows the user to return to the previous screen using `navController.popBackStack()`.
- ☐ Integrate the navigation host into a Scaffold that includes the AppBar.

☐ References:

- ☐ App Bars in Jetpack Compose
- ☐ Navigation Compose

2 Navigation Drawer with Deep Links

Enhance your application from Section 1 by implementing a `ModalNavigationDrawer` that provides direct access to the screens as well as additional deep links.

Hint: Deep Links & Intents

To launch external applications (Web, Maps, PDF), use Android **Intents** with the appropriate **Action** (e.g., `ACTION_VIEW`). Ensure your `AndroidManifest.xml` is configured correctly if you want the app to handle incoming links, or use a **Context** to start an activity for outgoing links.

☐ **Drawer Implementation:**

- ☐ Implement a `ModalNavigationDrawer` using Material3's `ModalNavigationDrawer` component.
- ☐ Populate the drawer with navigation items that open the screens.
- ☐ Add a visual divider (`HorizontalDivider`) to separate standard navigation from deep links.

☐ **Deep Links:**

- ☐ **Web Link:** Open a specified webpage in a browser using an `ACTION_VIEW` intent.
- ☐ **Map Link:** Display a set of coordinates; tapping it should open a navigation application (e.g., Google Maps).
- ☐ **File Link:** Open a local PDF document in an external PDF reader app. (Ensure the file is stored on the emulator/device).

☐ **Integration and Testing:**

- ☐ Ensure the `ModalNavigationDrawer` is accessible from every screen.
- ☐ Verify that the deep links correctly trigger the external applications and handle potential errors (e.g., no PDF reader installed).

☐ **References:**

- ☐ Common Intents (Android Developers)
- ☐ Navigation Drawers in Compose

3 CompositionLocals vs. Explicit Parameter Passing

Develop and compare two implementations of a user profile UI using Jetpack Compose and Material3. The user profile should display user data (name, address, and phone number) and a greeting message. One implementation must use *CompositionLocals* to manage styling properties (such as text alpha and text style), while the other implementation should pass these styling values explicitly as parameters.

Core Concept: CompositionLocal vs. Prop Drilling

CompositionLocal is a tool for passing data through the composition tree implicitly. It is ideal for "cross-cutting concerns" like themes or user settings. In contrast, "Prop Drilling" (passing parameters explicitly) ensures clear data flow but can lead to cumbersome function signatures in deep hierarchies.

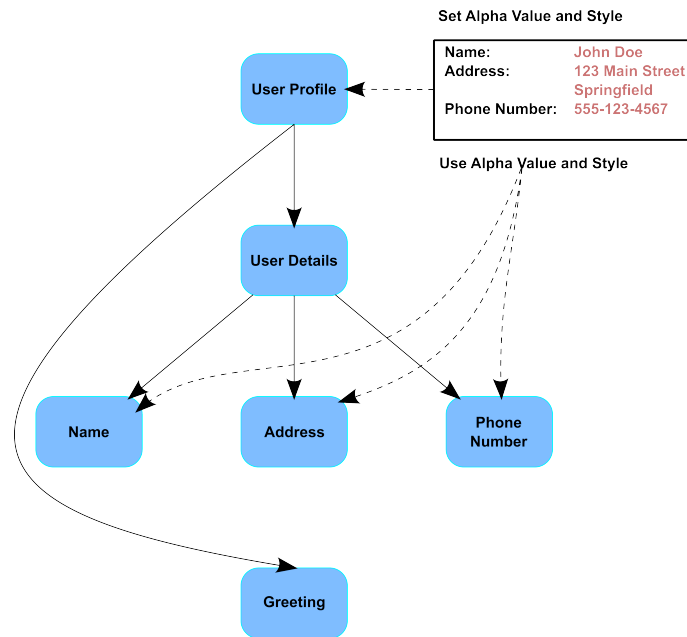


Figure 1: Comparison of data flow: Implicit (CompositionLocal) vs. Explicit Parameter Passing.

□ Implementation with CompositionLocals:

- Define a **CompositionLocal** for text alpha (default: 1.0f) and text style (e.g., **TextStyle.Default**).
- Create a **UserProfile** composable that displays the user's name, address, and phone number.
- Use **CompositionLocalProvider** to override the default values: apply a reduced alpha (e.g., 0.3f) and a distinct style (e.g., red, bold, 16.sp).

- ☐ Include a **Greeting** composable that displays a welcome message and the user's name, using the style/alpha provided via the composition local.
- ☐ **Implementation without CompositionLocals:**
 - ☐ Re-implement the **UserProfile** composable by passing alpha and style explicitly as parameters to all child composables (**Name**, **Address**, **PhoneNumber**, **Greeting**).
 - ☐ Ensure both implementations produce the identical visual result.
- ☐ **Comparison and Testing:**
 - ☐ Compare the two approaches regarding code clarity, maintainability, and scalability.
 - ☐ Reflect: When should **CompositionLocal** be preferred over explicit parameter passing?
- ☐ **References:**
 - ☐ Locally scoped data with **CompositionLocal**

4 LaunchedEffect

*Enhance your application by implementing a countdown timer that utilizes **LaunchedEffect** for coroutine-based time control. The timer should support pause/resume functionality and reset.*

Hint: LaunchedEffect & Keys

Remember that **LaunchedEffect** restarts whenever its **key** changes. To pause and resume the timer effectively, consider carefully what you pass as a key. Alternatively, manage the coroutine scope or the **isActive** state within your logic to handle the pause/resume behavior without constantly cancelling the effect.

- ☐ **Countdown Functionality:**
 - ☐ Display the current countdown value in the center of the screen (starting from 10).
 - ☐ Use **LaunchedEffect** to decrement the timer by 1 every second.
 - ☐ Automatically stop the countdown when it reaches 0.
 - ☐ Print a log message "Timer started" when the countdown begins.
 - ☐ Print a log message "Timer finished" when the countdown reaches 0.
- ☐ **Timer Controls:**

- ☐ Add a toggle button labeled "Start" or "Pause" depending on the current state of the timer.
- ☐ When the timer is paused, the countdown should stop; when resumed, it should continue from the current value.
- ☐ Add a separate "Reset" button to reset the timer back to 10 and stop it.
- ☐ **State Management:**
 - ☐ Use `remember` and `mutableStateOf` to store both the countdown value and the running/paused state.
 - ☐ Ensure the UI properly updates as the timer counts down, pauses, resumes, and resets.
- ☐ **References:**
 - ☐ LaunchedEffect — Jetpack Compose Side-Effects
 - ☐ State in Jetpack Compose
 - ☐ Kotlin Coroutines